

LAB PROGRAMS



Contents

- Install and set up Python and essential libraries like NumPy and pandas.
- Introduce scikit-learn as a machine learning library.
- Install and set up scikit-learn and other necessary tools.
- Write a program to Load and explore the dataset of .CSV and excel files using pandas.
- Write a program to Visualize the dataset to gain insights using Matplotlib or Seaborn by plotting scatter plots, bar charts.
- Write a program to Handle missing data, encode categorical variables, and perform feature scaling.
- Write a program to implement a k-Nearest Neighbours (k-NN) classifier using scikitlearn and Train the classifier on the dataset and evaluate its performance.
- Write a program to implement a linear regression model for regression tasks and Train the model on a dataset with continuous target variables.
- Write a program to implement a decision tree classifier using scikit-learn and visualize the decision tree and understand its splits.
- Write a program to Implement K-Means clustering and Visualize clusters.

Program 1 Install and set up Python and essential libraries like NumPy and pandas.

Setting up Python and essential libraries on a Windows system for machine learning involves a series of straightforward steps that prepare the environment for data analysis and algorithm development. By installing Python along with NumPy and Pandas, users can handle a wide array of data manipulation tasks efficiently.

Follow the below steps to set up Python and essential libraries such as NumPy and Pandas for machine learning on Windows.

Step 1: Install Python

Download Python: Go to the official Python website at python.org, navigate to the "Downloads" section, and download the latest version for Windows. Choose the executable installer.

Install Python: Execute the downloaded file. It is crucial to check the box labeled "Add Python 3.x to PATH" at the start of the installation wizard. Select "Customize installation" and ensure all options, including "pip", are selected. In the "Advanced Options," choose "Install for all users" and set the installation path to C:\Python. Proceed by clicking "Install".

Step 2: Install PIP

PIP generally comes installed with Python 3.4 and later. To confirm its installation, open Command Prompt and execute:

```
pip --version
```

If pip is not installed or if we need to update it, we can use the following command to install or upgrade pip:

```
python -m ensurepip --upgrade
```

After installation, we can verify that pip is installed correctly by running:

```
python -m pip --version
```

Step 3: Workspace Creation:

A dedicated directory for machine learning projects should be created for organizational clarity. This can be set up using Command Prompt:

```
C:\> mkdir C:\ML_Projects
```

```
C:\> cd C:\ML_Projects
```

Step 4: Creating a Virtual Environment

It is recommended to work in an isolated environment to manage dependencies more effectively and avoid conflicts between projects. The virtual environment tool should be installed and a new environment created:

```
C:\> pip install virtualenv # Install virtualenv
```

```
C:\> virtualenv ml_env # Create a new virtual environment named ml_env
```

```
C:\> ml_env\Scripts\activate # Activate the virtual environment
```

While activated, any packages installed using pip will only affect this environment.

To exit the virtual environment, simply run: **deactivate**

Step 5: Installing Necessary Tools:

Essential Libraries: Libraries such as Jupyter, NumPy, pandas, Matplotlib, and Scikit-Learn should be installed if they are not already present. These can be installed using pip, which is Python's package manager. Open Command Prompt and enter the following commands:

```
(ml_env) C:\> python -m pip install --upgrade pip  
(ml_env) C:\> pip install matplotlib numpy pandas scikit-learn
```

Program 2 Introduce scikit-learn as a machine learning library.

Scikit-learn is a popular open-source machine learning library in Python that offers a comprehensive set of tools and algorithms for data analysis, modeling, and machine learning tasks. It is built on foundational libraries like NumPy, SciPy, and Matplotlib. Scikit-learn provides a user-friendly and efficient framework for both beginners and experts in the field of data science.

Some key points to introduce scikit-learn as a machine learning library:

1. **Comprehensive Machine Learning Library:** Scikit-learn offers a wide range of machine learning algorithms and tools for various tasks such as classification, regression, clustering, dimensionality reduction, and more.
2. **User-Friendly and Easy to Use:** It is designed with a user-friendly interface and simple syntax, making it accessible for both beginners and experienced machine learning practitioners.
3. **Integration with Scientific Computing Libraries:** Scikit-learn integrates well with other scientific computing libraries in Python such as NumPy, SciPy, and Matplotlib, providing a powerful environment for machine learning tasks.
4. **Extensive Documentation and Community Support:** The library comes with comprehensive documentation, tutorials, and examples to help users understand and implement machine learning algorithms effectively. Additionally, there is a vibrant community around scikit-learn that provides support and contributions.
5. **Efficient Implementation of Algorithms:** Scikit-learn is built on top of NumPy, SciPy, and Cython, which allows for efficient implementation of machine learning algorithms and scalability to large datasets.
6. **Support for Model Evaluation and Validation:** The library provides tools for model evaluation, hyperparameter tuning, cross-validation, and performance metrics, enabling users to assess and improve the quality of their machine learning models.
7. **Flexibility and Customization:** Scikit-learn offers flexibility for customization and parameter tuning, allowing users to adapt algorithms to their specific requirements and datasets.
8. **Wide Adoption and Industry Usage:** Due to its ease of use, performance, and versatility, scikit-learn is widely adopted in academia, research, and industry for various machine learning applications.

Overall, scikit-learn is a powerful and versatile machine learning library in Python that empowers users to build and deploy machine learning models efficiently for a wide range of tasks and applications.

Program 3 Install and set up scikit-learn and other necessary tools.

Same as Program 1.

Program 4 Write a program to Load and explore the dataset of CSV and excel files using pandas.

Step 1: Creating CSV and Excel Files with Dummy Data

- **Create CSV File:** Open a text editor like Notepad or any other code editor. Enter the following data
Name, Age, Score
Srikanth, 28, 85
Snigdha, 22, 78
Mary, 31, 92

Save this file as **sample_data.csv** in the C:\ML_Projects directory.

- **Create Excel File:** We can use Microsoft Excel or Google Sheets to create this file. Enter the below data:

Name	Course	Sem
Rajesh	BCA	1
Ramesh	BCA	2
Swati	BCOM	1
Florina	BCOM	3
Pooja	BBA	2
Raghu	BBA	4

Save this file as **sample_data.xlsx** in the C:\ML_Projects directory.

Step 2: Python Code to Load and Explore the Data

import pandas as pd

Define the file paths

csv_file_path = 'C:\\ML_Projects\\sample_data.csv'

excel_file_path = 'C:\\ML_Projects\\sample_data.xlsx'

Load the CSV file

data_csv = pd.read_csv(csv_file_path)

print("CSV File Data:")

print(data_csv)

Load the Excel file

data_excel = pd.read_excel(excel_file_path)

print("\nExcel File Data:")

print(data_excel)

Basic Data Exploration

print("\nData Descriptions:")

print("CSV Data Description:")

print(data_csv.describe())

print("\nExcel Data Description:")

print(data_excel.describe())

```
# Displaying data types
print("\nData Types in CSV File:")
print(data_csv.dtypes)

print("\nData Types in Excel File:")
print(data_excel.dtypes)
```

Output**CSV File Data:**

	Name	Age	Score
0	Srikanth	28	85
1	Snigdha	22	78
2	Mary	31	92

Excel File Data:

	Name	Course	Sem
0	Rajesh	BCA	1
1	Ramesh	BCA	2
2	Swati	BCOM	1
3	Florina	BCOM	3
4	Pooja	BBA	2
5	Raghu	BBA	4

Data Descriptions:**CSV Data Description:**

	Age	Score
count	3.000000	3.0
mean	27.000000	85.0
std	4.582576	7.0
min	22.000000	78.0
25%	25.000000	81.5
50%	28.000000	85.0
75%	29.500000	88.5
max	31.000000	92.0

Excel Data Description:

	Sem
count	6.000000
mean	2.166667
std	1.169045
min	1.000000
25%	1.250000
50%	2.000000
75%	2.750000
max	4.000000

Data Types in CSV File:

Name	object
Age	int64
Score	int64
dtype:	object

Data Types in Excel File:

Name	object
Course	object
Sem	int64
dtype:	object

Explanation

- **Importing pandas:** The script begins by importing the pandas library, which is essential for data manipulation and analysis.
- **Loading Data:** The `pd.read_csv()` function is used to load data from the CSV file, and `pd.read_excel()` is for loading data from the Excel file.
- **Printing Data:** The script prints the data loaded from both files to ensure they are read correctly.
- **Data Exploration:** The `.describe()` method provides a statistical summary of the numerical columns, which helps quickly assess data distribution, count, mean, std, min, max, and percentiles.
- **Data Types:** The `.dtypes` attribute of the DataFrame is used to print the data types of each column, helping to confirm data formats and identify any potential issues with type mismatches.

Program 5

Write a program to Visualize the dataset to gain insights using Matplotlib by plotting scatter plots, bar charts.

Step 1: Create the CSV File :

Create a CSV file with below data of student study hours and exam scores: Save this file as **study_data.csv**.

```
Student ID,Study Hours,Exam Score
1,5,82
2,2,48
3,8,90
4,1,35
5,3,50
6,4,66
7,9,95
8,6,75
9,7,88
10,0.5,30
11,10,96
12,0,20
13,12,98
```

Step 2: Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the data
data = pd.read_csv('C:\\ML_Projects\\study_data.csv')

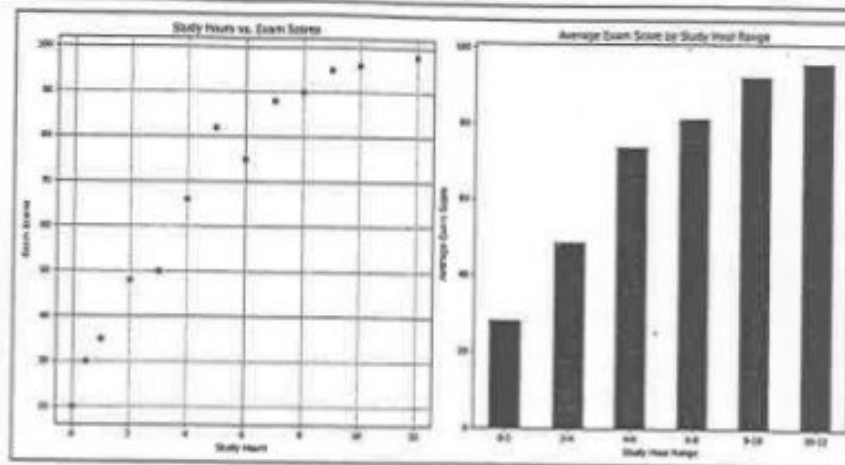
# Scatter plot of Study Hours vs. Exam Scores
plt.figure(figsize=(14, 7))
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st subplot
plt.scatter(data['Study Hours'], data['Exam Score'], color='dodgerblue', edgecolor='k',
            alpha=0.7)
plt.title('Study Hours vs. Exam Scores')
plt.xlabel('Study Hours')
plt.ylabel('Exam Scores')
plt.grid(True)

# Bar chart of Average Exam Score by Study Hour Range
# Creating bins for study hour ranges
bins = [0, 2, 4, 6, 8, 10, 12]
labels = ['0-2', '2-4', '4-6', '6-8', '8-10', '10-12']
data['Study Hour Range'] = pd.cut(data['Study Hours'], bins=bins, labels=labels,
                                right=False)
grouped_data = data.groupby('Study Hour Range')['Exam Score'].mean()
```

```
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd subplot
grouped_data.plot(kind='bar', color='salmon')
plt.title('Average Exam Score by Study Hour Range')
plt.xlabel('Study Hour Range')
plt.ylabel('Average Exam Score')
plt.xticks(rotation=0) # Keep the category labels horizontal

plt.tight_layout() # Adjust subplots to fit into figure area.
plt.show()
```

Output



Explanation

- 1. Data Loading:** The script uses pandas to load a CSV file containing students' study hours and exam scores from a specified path.
- 2. Scatter Plot:** Matplotlib is employed to create a scatter plot, plotting 'Study Hours' against 'Exam Scores' to visually explore the relationship between these variables.
- 3. Bar Chart Setup:** The data is categorized into bins based on study hours using `pd.cut()`. It then calculates the average exam score for each category and uses this data to generate a bar chart showing average scores by study hour range.
- 4. Visualization Configuration:** Both plots are configured in a single figure, with clear titles and labels for axes, using `plt.subplot()` to arrange them side by side for easy comparison.
- 5. Display:** The script concludes with `plt.show()` to display the configured plots, providing insights into how study time correlates with academic performance through both detailed and aggregated views.

Program 6

Write a program to Handle missing data, encode categorical variables, and perform feature scaling.

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Create dummy data
data = {
```

```

'Age': [25, 30, None, 28, 35],
'Gender': ['Female', 'Male', 'Male', 'Female', 'Male'],
'Income': [50000, 60000, 45000, None, 70000]
}
df = pd.DataFrame(data)

# Handling missing data
imputer = SimpleImputer(strategy='mean')
df[['Age', 'Income']] = imputer.fit_transform(df[['Age', 'Income']])

# Print data after handling missing values
print("Data after handling missing values:")
print(df)

# Encoding categorical variables
encoder = OneHotEncoder()
encoded_data = encoder.fit_transform(df[['Gender']]).toarray()

# Print data after categorical encoding
encoded_df = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(['Gender']))
print("\nData after categorical encoding:")
print(encoded_df)

# Feature scaling
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[['Age', 'Income']])

# Print data after feature scaling
scaled_df = pd.DataFrame(scaled_data, columns=['Scaled Age', 'Scaled Income'])
print("\nData after feature scaling:")
print(scaled_df)

```

Output

Data after handling missing values:

	Age	Gender	Income
0	25.0	Female	50000.0
1	30.0	Male	60000.0
2	29.5	Male	45000.0
3	28.0	Female	56250.0
4	35.0	Male	70000.0

Data after categorical encoding:

	Gender_Female	Gender_Male
0	1.0	0.0
1	0.0	1.0
2	0.0	1.0
3	1.0	0.0
4	0.0	1.0

Data after feature scaling:

	Scaled Age	Scaled Income
0	-1.382164	-0.727778
1	0.153574	0.436667
2	0.000000	-1.310001
3	-0.460721	0.000000
4	1.689312	1.601112

Explanation

1. Data Preparation:

The code creates a dummy dataset with columns 'Age', 'Gender', and 'Income' containing numerical and categorical data.

It uses the pandas library to create a DataFrame from the dummy data, which will be used for further processing.

2. Handling Missing Data:

The code uses SimpleImputer from sklearn.impute to fill missing values in the 'Age' and 'Income' columns with the mean of each respective column.

This step ensures that the dataset is ready for further processing without missing values affecting the analysis.

3. Categorical Encoding:

The code utilizes OneHotEncoder from sklearn.preprocessing to encode the categorical variable 'Gender' into a one-hot encoded format.

The fit_transform method is applied to convert the categorical data into a numerical representation suitable for machine learning algorithms.

4. Printing Data after Categorical Encoding:

After encoding the categorical variable 'Gender', the code creates a DataFrame encoded_df to display the data in its one-hot encoded form.

The get_feature_names_out method is used to retrieve the feature names for the encoded columns based on the original categorical variable.

5. Feature Scaling:

The code uses StandardScaler from sklearn.preprocessing to standardize the numerical columns 'Age' and 'Income'.

Standardization ensures that all numerical attributes have a mean of 0 and a standard deviation of 1, which can improve the performance of certain machine learning algorithms.

Program 7

Write a program to implement a k-Nearest Neighbours (k-NN) classifier using scikitlearn and Train the classifier on the dataset and evaluate its performance.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Dummy student data: exam score 1, exam score 2, pass/fail (features)
X = np.array([[80, 75], [95, 90], [60, 50], [45, 30], [30, 40], [85, 95], [70, 60], [50, 55],
              [40, 45], [60, 70]])
y = np.array([1, 1, 0, 0, 0, 1, 1, 0, 0, 1]) # Binary classes for demonstration

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the k-NN classifier with k=3
knn = KNeighborsClassifier(n_neighbors=3)

# Train the classifier on the training data
knn.fit(X_train, y_train)

# Evaluate the classifier's performance
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy on the test set: {:.2f}".format(accuracy))

# Take user input for exam scores
exam_score1 = float(input("Enter Exam Score 1: "))
exam_score2 = float(input("Enter Exam Score 2: "))

# Prepare the user input for prediction
user_input = np.array([[exam_score1, exam_score2]])

# Use the trained k-NN classifier to predict the outcome
predicted_outcome = knn.predict(user_input)

if predicted_outcome[0] == 1:
    print("Based on the exam scores provided, the student is predicted to pass.")
else:
    print("Based on the exam scores provided, the student is predicted to fail.")
```

Output**Output 1:**

Accuracy on the test set: 1.00

Enter Exam Score 1: 45

Enter Exam Score 2: 50

Based on the exam scores provided, the student is predicted to fail.

Output 2:

Accuracy on the test set: 1.00

Enter Exam Score 1: 75

Enter Exam Score 2: 89

Based on the exam scores provided, the student is predicted to pass.

Explanation**1. Data Preparation:**

The code initializes a numpy array `X` with exam scores as features and `y` with binary pass/fail labels. It represents a simple dataset where each row corresponds to a student's exam scores and pass/fail outcome.

2. Model Training and Evaluation:

It splits the data into training and testing sets using `train_test_split` with a test size of 20% and a random seed for reproducibility.

The code initializes a K-Nearest Neighbors (KNN) classifier with `n_neighbors=3` and trains it on the training data (`X_train, y_train`).

The model's performance is evaluated by predicting on the test set (`X_test`) and calculating the accuracy using `accuracy_score`.

3. User Interaction:

The code prompts the user to input exam scores for a new student using `input()` function.

It prepares the user input as a numpy array `user_input` to make a prediction using the trained KNN classifier.

4. Prediction and Output:

The code predicts the outcome (pass/fail) for the new student based on the input exam scores using the trained KNN classifier.

It then prints a message indicating whether the student is predicted to pass or fail based on the model's prediction.

Program 8

Write a program to implement a linear regression model for regression tasks and Train the model on a dataset with continuous target variables.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Dummy house price prediction data: features (house size, number of bedrooms) and
# target variable (house price)
X = np.array([[1000, 2], [1500, 3], [1200, 2], [1800, 4], [900, 2], [2000, 3]])
y = np.array([300000, 400000, 350000, 500000, 280000, 450000])
```

```
# Initialize the Linear Regression model
model = LinearRegression()

# Train the model on the dataset
model.fit(X, y)

# Take input from the user for new house data
size = float(input("Enter the size of the house in sqft: "))
bedrooms = int(input("Enter the number of bedrooms: "))
new_data = np.array([[size, bedrooms]])

# Predict the price for the new house data
predicted_price = model.predict(new_data)

# Print the predicted price for the new house data
print("Predicted price for a house with size {} sqft and {} bedrooms: Rs. {:.2f}".format(size, bedrooms, predicted_price[0]))
```

Output

```
Enter the size of the house in sqft: 1600
Enter the number of bedrooms: 3
Predicted price for a house with size 1600.0 sqft and 3 bedrooms: Rs.418163.93
```

Explanation

1. Model Training and Prediction:

The code initializes a Linear Regression model and trains it on the dummy house price prediction data provided in the arrays X (features - house size and number of bedrooms) and y (target variable - house price).

After training the model, it takes input from the user for new house data (size and number of bedrooms) to predict the price for a new house.

The model then predicts the price for the new house data using the predict() method and stores the result in predicted_price.

2. User Input and Output:

The code prompts the user to enter the size of the house in square feet and the number of bedrooms using the input() function.

It converts the user input into a NumPy array new_data to match the format expected by the model for prediction.

Finally, it prints the predicted price for the new house data input by the user in the format "Predicted price for a house with size [size] sqft and [bedrooms] bedrooms: Rs.[predicted_price]".

```
from sklearn.linear_model import LinearRegression
```

```
# Dummy house price prediction data: features (house size, number of bedrooms) and  
target variable (house price)
```

```
X = np.array([[1000, 2], [1500, 3], [1200, 2], [1800, 4], [900, 2], [2000, 3]])
```

```
y = np.array([300000, 400000, 350000, 500000, 280000, 450000])
```

Program 9

Write a program to implement a decision tree classifier using scikit-learn and visualize the decision tree and understand its splits.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.tree import export_text
import matplotlib.pyplot as plt

# Custom dummy data for fruit classification
# Features: [Weight, Texture] -> Target: [Fruit Type]
X = np.array([[150, 0], [170, 1], [120, 0], [140, 1], [200, 1], [130, 0]])
y = np.array(['Apple', 'Orange', 'Apple', 'Orange', 'Melon', 'Apple'])

# Initialize the Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X, y)

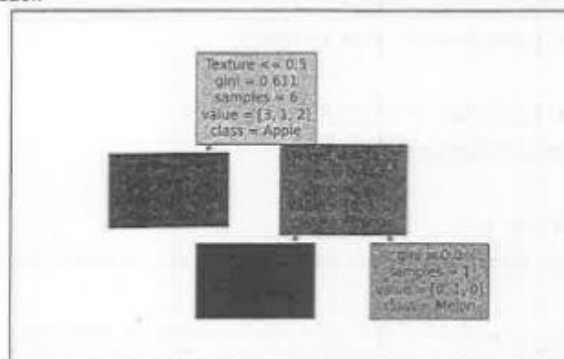
# Visualize the Decision Tree splits
tree_rules = export_text(clf, feature_names=['Weight', 'Texture'])
print("Decision Tree Classifier Rules:\n", tree_rules)

# Plot the Decision Tree
plt.figure(figsize=(10, 6))
plot_tree(clf, filled=True, feature_names=['Weight', 'Texture'], class_names=np.unique(y))
plt.show()
```

Output

Decision Tree Classifier Rules:

```
|--- Texture <= 0.50
|   |--- class: Apple
|--- Texture > 0.50
|   |--- Weight <= 185.00
|   |   |--- class: Orange
|   |--- Weight > 185.00
|   |   |--- class: Melon
```



Explanation**1. Data Preparation:**

The code defines a custom dummy dataset for fruit classification with features representing 'Weight' and 'Texture' of fruits and the target variable 'Fruit Type' (e.g., Apple, Orange, Melon).

The features and target labels are stored in NumPy arrays X and y, respectively.

2. Decision Tree Classifier Initialization:

A Decision Tree Classifier is initialized with `random_state=42` to ensure reproducibility of results.

The classifier is then trained on the custom dummy dataset using the `fit()` method.

3. Visualization of Decision Tree Splits:

The `export_text` function is used to generate text-based rules of the Decision Tree Classifier based on the features provided.

These rules provide insights into how the Decision Tree makes splits based on the 'Weight' and 'Texture' features to classify different types of fruits.

4. Plotting the Decision Tree:

The `plot_tree` function is utilized to visualize the Decision Tree structure graphically.

The Decision Tree is displayed with filled nodes, and the feature names ('Weight', 'Texture') and class names (unique fruit types) are specified for better interpretation.

5. Displaying the Decision Tree Visualization:

A Matplotlib figure is created with a specific size to accommodate the Decision Tree plot.

The Decision Tree visualization is shown using `plt.show()`, allowing us to observe the tree structure and decision-making process visually.

Program 10 Write a program to Implement K-Means clustering and Visualize clusters.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

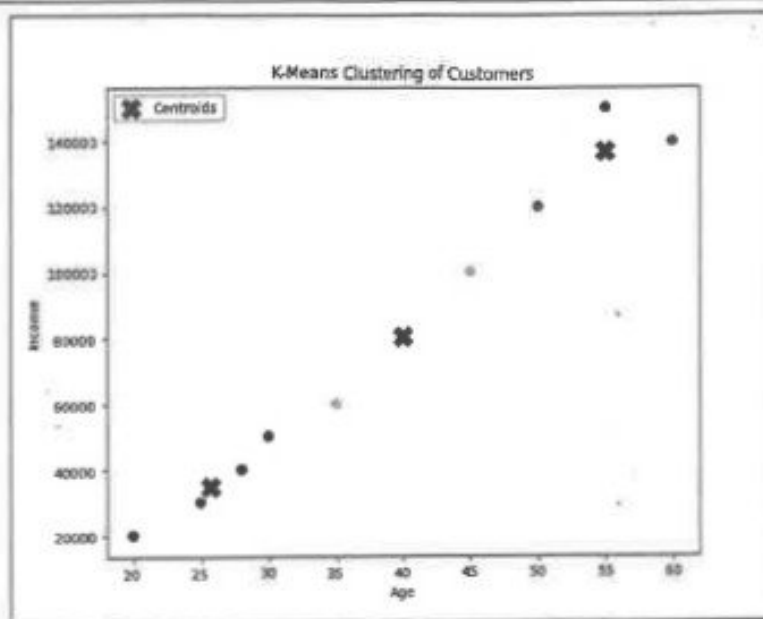
# Generate dummy customer data (Age, Income)
X = np.array([[30, 50000], [35, 60000], [40, 80000], [25, 30000], [45, 100000],
              [20, 20000], [50, 120000], [55, 150000], [60, 140000], [28, 40000]])

# Initialize K-Means with 2 clusters
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)

# Get cluster labels and cluster centers
labels = kmeans.labels_
centers = kmeans.cluster_centers_
```

```
# Visualize the clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8)
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='x', label='Centroids')
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('K-Means Clustering of Customers')
plt.legend()
plt.show()
```

Output



Explanation

1. Dummy Data Generation:

The program generates dummy customer data with features representing 'Age' and 'Income' of customers.

K-Means Clustering:

K-Means clustering is applied to the customer data with $n_clusters=3$ to create 3 clusters.

The algorithm assigns each data point to one of the clusters based on the similarity of features.

2. Visualization:

The clusters are visualized using a scatter plot where each point represents a customer.

Different clusters are distinguished by colors, and cluster centers (centroids) are marked in red.

3. Plot Interpretation:

The plot helps visualize how customers are grouped into clusters based on their 'Age' and 'Income'.

Centroids represent the center of each cluster, showing the average 'Age' and 'Income' values for customers in that cluster.